

ANÁLISE EXPERIMENTAL DE ALGORITMOS: BUBBLE SORT & QUICK SORT

Lucas Ângelo Câmara da Silva

Instituto Politécnico, Universidade do Estado do Rio de Janeiro

Disciplina: Projeto e Análise de Algoritmos

Professor: Prof^ª. Dr^ª. Camila Martins Saporetti

Junho, 2025

Resumo

Uma análise e comparação didática de dois algoritmos ordinários de ordenação: Bubble Sort (desempenho consistentemente inferior) & Quick Sort (desempenho altamente eficiente na maioria dos cenários).

Palavras-chave

Algoritmos, Ordenação, Complexidade, Análise de Algoritmos, Bubble Sort, Quick Sort.

INTRODUÇÃO

Os algoritmos de ordenação são algoritmos determinísticos muito utilizados no aprendizado de computação por demonstrarem a interferência de fluxos de controles na performance de programas. Este estudo foca na comparação tamanho da entrada x tempo de execução de dois algoritmos diferentes: bubble sort - move elementos da esquerda para a direita até que o elemento movido continuar sendo menor que o que ele ultrapassou - e quick sort - *divide e conquista* o vetor ao redor de um pivô trocando os elementos grandes do lado esquerdo pelos elementos menores do lado direito (BIGGAR; GREGG, 2005).

METODOLOGIA

Algoritmos

- Bubble Sort

Funciona operando da esquerda para a direita, transportando elementos como se fossem bolhas através do vetor, prosseguindo para o próximo assim que ele passa de um elemento que é menor que ele mesmo. É dito como uma ordenação elementar, pois para cada elemento a ser ordenado, todos os outros elementos são verificados.

- Quick Sort

Seu principal fator é um pivô, um elemento - muitas vezes o elemento do meio, mas que também pode ser o elemento do início, fim ou um aleatório - que serve de âncora que verifica que os valores a sua esquerda sejam menores que ele e os valores a sua direita sejam maiores que ele, repartindo os dois lados do vetor e executando a operação recursivamente até o mesmo estar ordenado. Esse é um algoritmo de divisão e conquista por quebrar o vetor em porções menores para ordená-lo.

Computador

Os testes, todos executados em single thread na versão convencional do Python 3.10 e em um laptop com um processador Intel® Pentium® N3710 com 4 núcleos de 1.60GHz cada, com 4GB de memória RAM e 6GB de memória Swap, com o sistema operacional Linux Mint 21.3 Cinnamon.

RESULTADOS

Vale ressaltar que todos os gráficos apresentados possuem flutuações no eixo de tempo. Esse ruído se deve ao funcionamento complexo dos sistemas operacionais e hardware modernos, que alternam entre diferentes processos devido à funcionalidade de multitasking e o transporte de memória entre o cache da CPU e a memória RAM, porém todos os gráficos ainda apresentam padrões dignos de serem estudados.

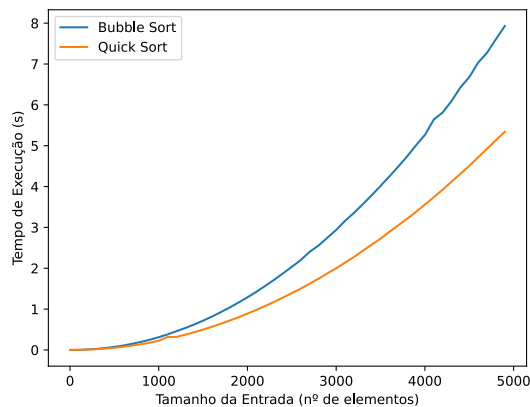


Gráfico 1: Ordenação de um vetor com números decrescentes

Este é o pior caso de que um algoritmo de ordenação pode encontrar, pois ele vai ter que ordenar todos os elementos do vetor. O Gráfico 1 mostra o Quick Sort performando 62.5% vezes mais rápido que o Bubble Sort performando, fato curioso devido a ambos os algoritmos possuírem a mesma notação de $O(n^2)$ no pior caso (BLACK, 1998), porém o Quick Sort performa melhor no teste devido as suas operações mais eficientes (em parte pela estratégia de *dividir e conquistar* o vetor).

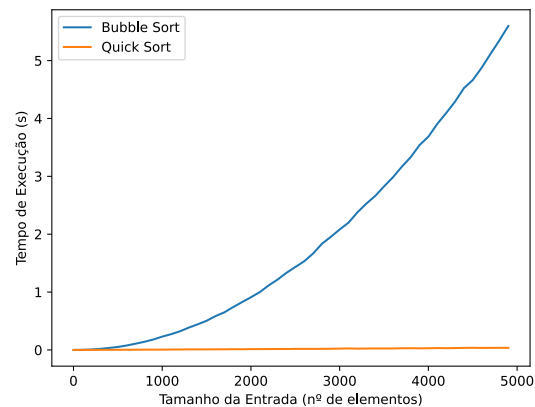


Gráfico 3: Ordenação de um vetor com números aleatórios

Porém é neste caso onde a diferença entre os dois fica mais notável, o teste de valores aleatórios, demonstrando um caso médio entre os dois anteriores. O Bubble Sort é **sempre ruim** em casos médios devido a ele performar com a mesma notação que em casos ruins de ordenação, com uma notação de $O(n^2)$, contra o Quick Sort, que demonstra comportamento **geralmente excelente** em casos médios por ele performar com a mesma notação que em casos *bons* de ordenação, com uma notação de $O(n \log n)$ (BLACK, 1998).

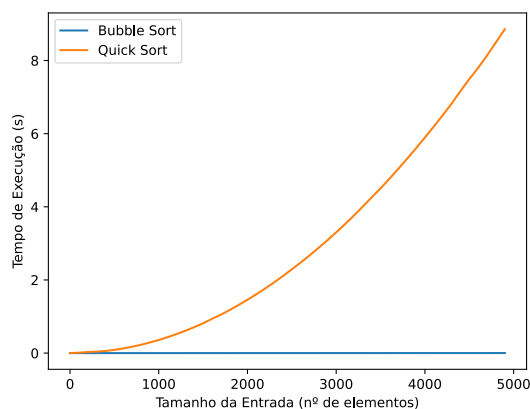


Gráfico 2: Ordenação de um vetor com números crescentes

Já este é o melhor caso que um algoritmo de ordenação pode encontrar, pois todos os números do vetor já estão ordenados. O Gráfico 2 mostra que o quicksort possui uma fraqueza neste caso, performando na notação $O(n^2)$ contra $O(n)$ (que aparenta ser $O(1)$ de tão rápido), fraqueza que se deve pelo mesmo fator que foi seu diferencial no teste anterior, a *divisão e a conquista*.

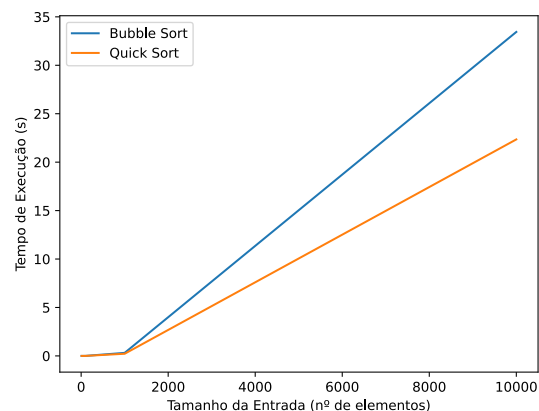


Gráfico 4: Ordenação periódica de um vetor com números decrescentes

Nos próximos 3 gráficos foi analisado casos mais extremos, de vetores de entrada 10, 100, 1.000 e 10.000 de forma periódica, onde é possível notar melhor as consequências das notações entre cada algoritmo, com o Gráfico 4 reforçando a diferença de tempo entre os dois.

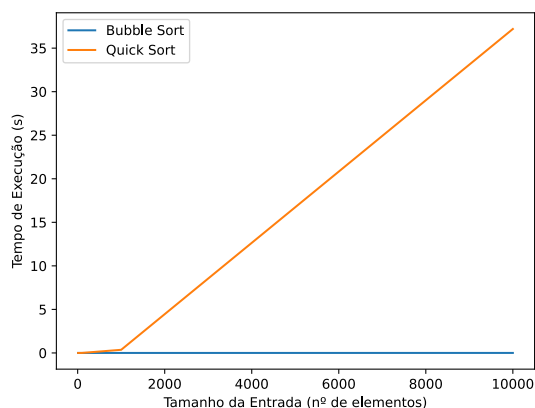


Gráfico 5: Ordenação periódica de um vetor com números crescentes

O Gráfico 5 enfatiza o malefício principal do Quick Sort com vetores já ordenados de tamanhos elevados.

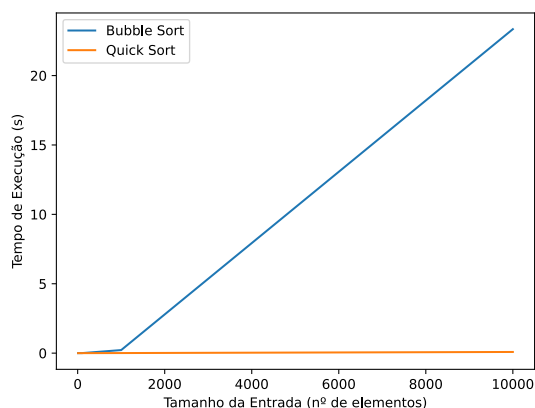


Gráfico 6: Ordenação periódica de um vetor com números aleatórios

E por ultimo, o Gráfico 6 enfatiza como o benefício do Quick Sort é notavel comparado com o Bubble Sort, com os dois quase se espelhando comparado com o Gráfico 5.

CONCLUSÃO

O Bubble Sort é simples em sua funcionalidade mas peca quando lida com vetores de complexidade média ou elevada, que é a grande maioria dos casos vistos no cotidiano, já o Quick Sort demonstra um comportamento exatamente oposto, sendo ruim apenas para vetores ordenados ou pouco ordenados, mas mesmo assim tomando pouco tempo de processamento nesses casos, sendo um dos algoritmos de ordenação mais populares e ja vindo embutidos em linguagens como o Java ("Arrays (Java Platform SE 8)," 1995).

REFERÊNCIAS

Arrays (Java Platform SE 8). Disponível em: <<https://docs.oracle.com/javase/8/docs/api/java/util/Arrays.html>>. Acesso em: may. 2025.

BIGGAR, P.; GREGG, D. **Sorting in the Presence of Branch Prediction and Caches: Fast Sorting on Modern Computers**. Dublin, Ireland: University of Dublin, Aug. 2005. Disponível em: <<https://www.cs.tcd.ie/publications/tech-reports/reports.05/TCD-CS-2005-57.pdf>>. Acesso em: may. 2025.

BLACK, P. **Dictionary of Algorithms and Data Structures**. Disponível em: <<https://xlinux.nist.gov/dads/>>. Acesso em: may. 2025.